Assignment 2, Part 3(a): CUBIC vs. Reno vs. BBR

Ayusha Patra Kabileswaran Krishna Kumar Vihaan Kodeboyina Millan Kumar

Summary

Across five public iperf3 servers and three repetitions, **BBR had the highest per-run mean goodput in all 15 destination–repetition comparisons** (78.1 Mbit/s mean vs. 52.1 for CUBIC and 43.6 for Reno). These overall means average successful tests; Reno has one fewer test, so the paired comparisons are more informative. BBR's destination-level mean RTT was 8.7–14.4 ms above CUBIC and 10.4–19.3 ms above Reno; its retransmitted-byte fraction averaged about 11%. CUBIC beat Reno in 13 of 14 matched comparisons. The traces are consistent with loss/recovery limiting CUBIC and Reno, but do not establish the sole bottleneck. A separate, shorter Docker rerun reported below changes the rankings, illustrating the limits of generalizing from these public paths.

Setup

Host	Ubuntu 24.04.5 VM (Multipass) on an Apple Silicon Mac, Wi-Fi uplink
Kernel	Linux `6.8.0-147-generic` (aarch64), BBRv1 via `modprobe tcp_bbr`
Command	`python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42`
Tests	60 s each, TCP_INFO sampled every 0.2 s, 3 repetitions × 5 destinations × 3 algorithms = 45

Part 3 reuses the five destinations from Part 1 (minimum RTT in parentheses): LeaseWeb US (48 ms), FortiSASE ES (105 ms), MilkyWan FR (109 ms), Multinet24 PL (130 ms), and Mauritius Telecom (288 ms). In each repetition, every destination runs all three algorithms back to back, and the order rotates: CUBIC→Reno→BBR, then Reno→BBR→CUBIC, then BBR→CUBIC→Reno. Each test is preceded by a 3 s pause, and refused tests are retried up to twice. One test (Reno, repetition 3, LeaseWeb) was refused every time, so 44 of 45 tests completed. The algorithm is set with `setsockopt(TCP_CONGESTION)` before connecting and verified with `getsockopt`.

Metrics (all from TCP_INFO on the data socket): goodput from 8Δ`bytes_acked`$/\Delta t$, using actual elapsed sampling time; smoothed RTT; excess-delay proxy = mean RTT − `min_rtt`; and loss proxy = retransmitted bytes / TCP bytes sent. The plots use retransmitted segments per interval. Retransmissions are not a direct packet-loss probability, and excess RTT does not isolate router queueing from wireless, host, or VM effects. Goodput summaries are arithmetic means of sampled rates; SD is the sample standard deviation of the per-run means. The minimum RTTs in destination labels describe representative path baselines, not identical RTTs in every algorithm/run.

Results

The plots use `148.230.45.213:30009`, the representative destination selected from Part 1 of this five-destination batch, with shared axis scales for all three algorithms. The combined report's earlier Question 2 figures use a separate ten-destination batch and `iperf.online.net:5209`; they must not be interpreted as the same run.

Table 1: Per destination: mean ± std. dev. of per-run goodput across repetitions, and means of the other metrics.

Destination (min RTT)	Algo	Goodput (Mbit/s)	RTT (ms)	Excess delay (ms)	Retrans. bytes	cwnd (segments)
LeaseWeb US	CUBIC	93.2 ± 1.4	77	28.7	0.04%	601
(48 ms)	Reno	85.7 ± 3.7	72	25.6	0.36%	538
	BBR	**144.5** ± 11.9	91	44.5	12.64%	2838
FortiSASE ES	CUBIC	48.6 ± 1.4	133	25.6	0.07%	588
(105 ms)	Reno	45.0 ± 1.6	129	23.3	1.01%	544
	BBR	**67.5** ± 1.4	141	35.7	9.08%	1961
MilkyWan FR	CUBIC	47.1 ± 2.0	132	24.1	0.05%	571
(109 ms)	Reno	45.0 ± 1.4	132	24.4	0.66%	546
	BBR	**72.8** ± 6.2	142	36.0	15.51%	2327
Multinet24 PL	CUBIC	41.5 ± 3.9	153	22.3	0.52%	605
(130 ms)	Reno	36.0 ± 2.1	155	23.8	0.54%	529
	BBR	**63.3** ± 5.3	168	37.5	11.43%	2360
Mauritius Tel.	CUBIC	30.0 ± 5.5	321	27.8	0.51%	834
(288 ms)	Reno	20.2 ± 2.5	316	22.4	1.00%	565
	BBR	**42.4** ± 1.9	332	39.3	7.39%	3207

Table 2: Earlier batch: minimum across per-run minima; median and p95 are means of per-run quantiles (not pooled quantiles); mean and sample SD summarize per-run means. Rates are Mbit/s. These statistics are reconstructible from the archived per-run summaries.

Destination	Algo	Runs	Min	Median	Mean	SD	P95
LeaseWeb US	CUBIC	3	4.02	96.63	93.25	1.43	133.79
LeaseWeb US	Reno	2	2.91	83.83	85.74	3.69	131.50
LeaseWeb US	BBR	3	0.26	141.02	144.54	11.92	278.66
Multinet24 PL	CUBIC	3	0.33	43.11	41.49	3.91	65.31
Multinet24 PL	Reno	3	0.43	34.94	36.03	2.11	61.12
Multinet24 PL	BBR	3	0.00	46.48	63.29	5.29	128.62
Mauritius Tel.	CUBIC	3	0.00	28.79	29.98	5.47	58.51
Mauritius Tel.	Reno	3	0.00	20.31	20.21	2.46	32.47
Mauritius Tel.	BBR	3	0.00	32.23	42.35	1.93	114.45
FortiSASE ES	CUBIC	3	0.44	51.88	48.62	1.44	68.61
FortiSASE ES	Reno	3	0.00	44.49	45.04	1.62	66.49
FortiSASE ES	BBR	3	0.00	58.14	67.51	1.44	125.01
MilkyWan FR	CUBIC	3	0.54	50.69	47.11	1.99	68.43
MilkyWan FR	Reno	3	0.50	43.84	44.98	1.36	67.94
MilkyWan FR	BBR	3	0.00	57.47	72.83	6.24	143.11

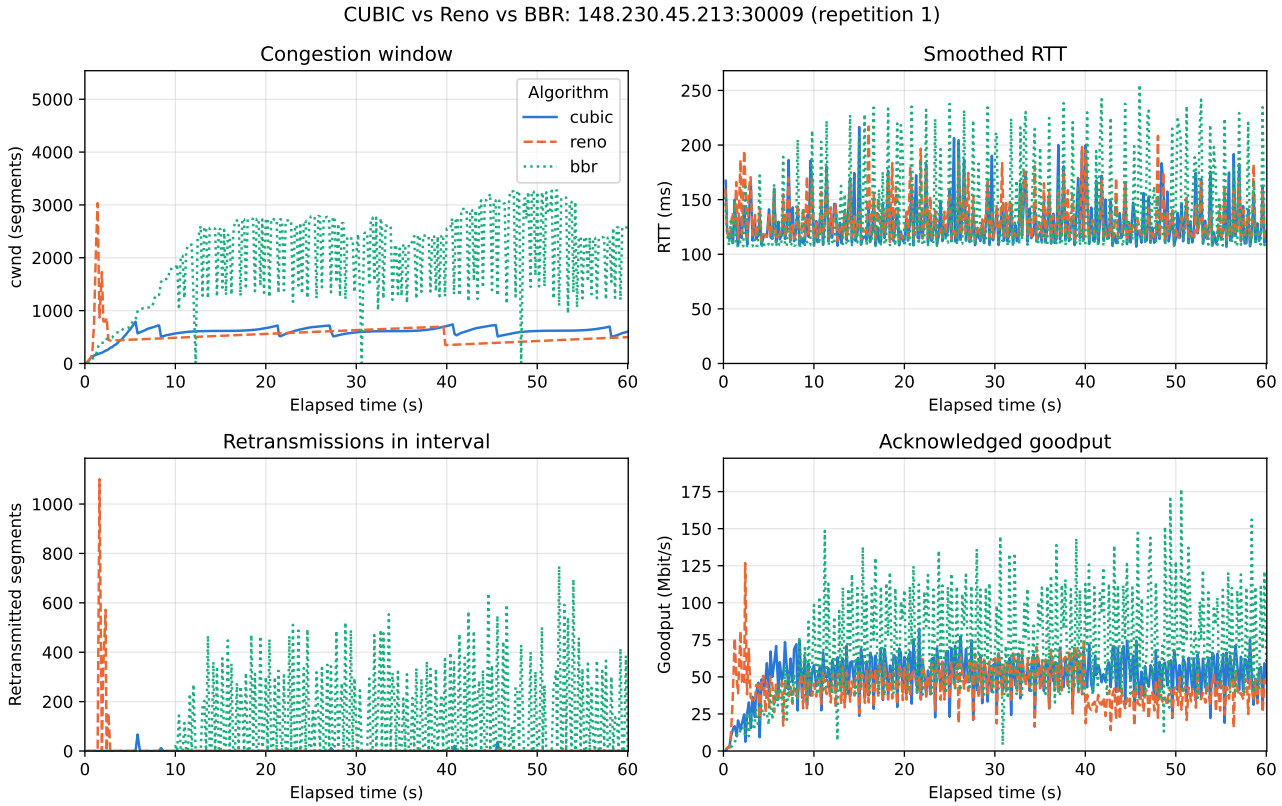


Figure 1: Time series at the representative destination (repetition 1).

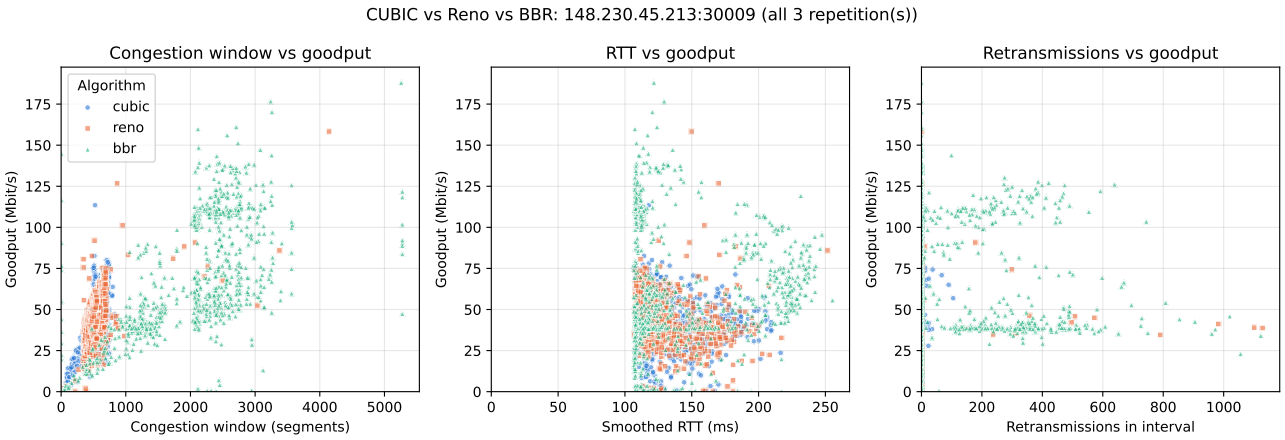


Figure 2: cwnd, RTT, and retransmissions vs. goodput (all 0.2 s samples, all three repetitions).

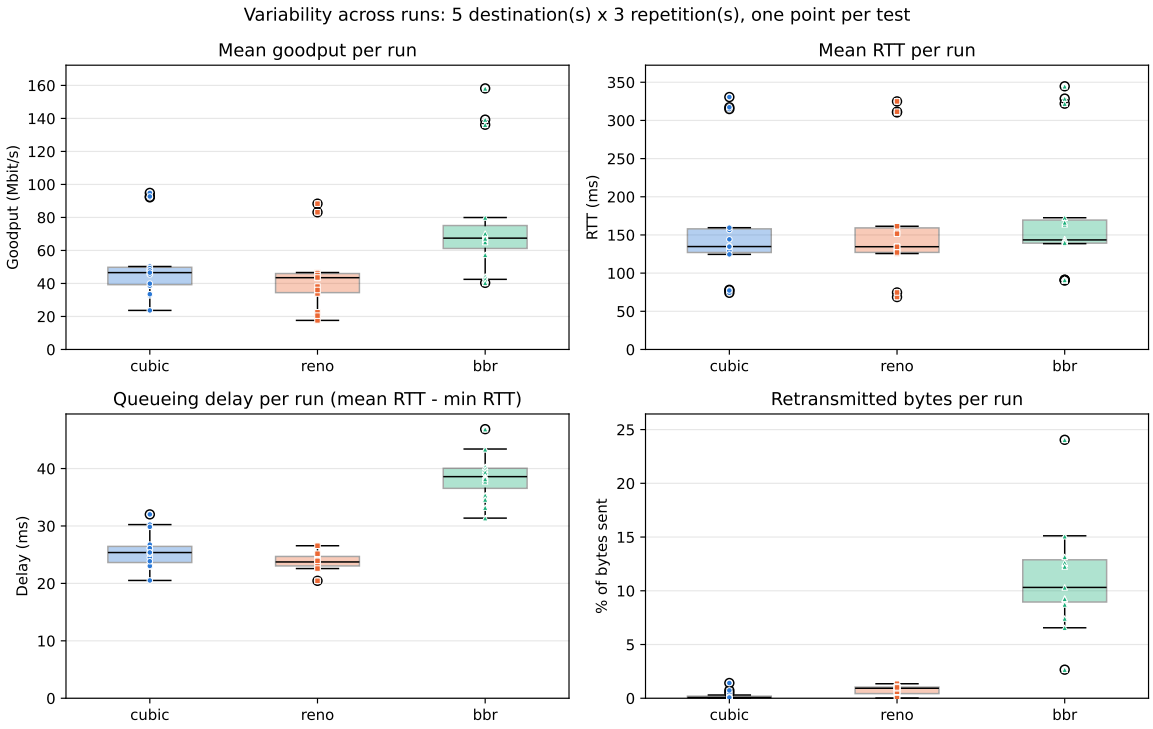


Figure 3: Run-to-run variability, one point per test (all destinations).

Analysis

Traces (Figure 1). After startup, Reno grows gradually and shows a marked window reduction near 40 s. CUBIC repeatedly reduces and regrows its window. Reno normally adds approximately one segment per RTT in congestion avoidance and targets a half-window threshold on a loss event; CUBIC uses time-based growth and a nominal 0.7 decrease factor. Neither reduces independently for every retransmitted packet in a burst. BBR uses a larger window and has repeated brief dips near 12, 31, and 48 s, consistent with ProbeRTT behavior; we did not log its internal mode to confirm this. Its sustained retransmission activity is visible in Figure 2.

Operating regime. On the four paths with baseline RTTs of 48–130 ms, CUBIC and Reno averaged roughly 530–610 segments. When congestion-window limited, the approximate supported rate is $8\,cwnd\,MSS/RTT$, with RTT in seconds and MSS in bytes. A similar window therefore supports less throughput at larger RTT. Window reductions and slow recovery are consistent with loss/recovery limitations. However, receiver windows, pacing, server load, and changing capacity can also limit goodput; retransmissions alone do not establish a congestion-loss rate or prove the bottleneck. BBRv1 primarily models delivery bandwidth and minimum RTT rather than applying Reno-style loss-driven growth, but still handles loss recovery and can respond to a detected policer.

Which algorithm is better where.

- **Throughput: BBR**, by 1.5–1.8× Reno on the 48–130 ms paths and 2.1× at 288 ms. On a long path, a loss-based algorithm needs more time to regrow a large window after backoff. This supports BBR for bulk throughput on these tested paths; it does not establish that their losses were non-congestive or that BBR wins generally.
- **CUBIC vs. Reno: CUBIC on high-BDP paths.** The gap was 1.05–1.15× up to 130 ms but

1.48× at 288 ms (window 834 vs. 565). CUBIC backs off less (to 70%, not 50%), and its growth depends on time since the last loss rather than on RTT, whereas Reno would need over 2 minutes to regrow 500 segments at 288 ms.

- **Delay and retransmissions: CUBIC and Reno.** Their grouped excess-delay proxies were about 22–29 ms. BBR had greater mean RTT on all five destinations. Retransmitted-byte percentages were 0.04–0.52% for CUBIC, 0.36–1.01% for Reno, and 7.39–15.51% for BBR. BBRv1's probing and larger window targets are consistent with these observations, but cwnd is a limit, not the measured bytes in flight, and we did not measure bottleneck buffer occupancy.
- **BDP.** The BDP implied by achieved goodput is about 0.5–0.9 MB on the shorter paths and up to 1.5 MB in an individual long-path run. The code computes median achieved goodput times minimum RTT, divided by eight for bytes. This is a goodput-based BDP proxy, not independently measured capacity BDP. Converting cwnd to bytes requires the recorded MSS. Thus the observed window differences support a throughput/RTT interpretation but do not prove underfilling or buffer overflow relative to the true path BDP.

We did not control buffer depth, capacity, or background traffic, so these tests do not isolate deep-buffer or uncongested-capacity regimes. BBR can have lower delay in other conditions; the separate rerun below demonstrates that the ranking is sensitive to the measurement setting.

Variability. On the same destination, results were consistent: the coefficient of variation of per-run goodput was 1.5–9.4% for 13 of 15 destination–algorithm pairs, and up to 18% on the long Mauritius path (Figure 3). Across destinations, goodput varied 3–4×, so the path matters more than the repetition. The ranking held regardless of test order: BBR was fastest in all 15 groups, and CUBIC beat Reno in 13 of 14 (the exception was a 0.9 Mbit/s difference).

Limitations. Public paths change between tests, and three repetitions is a small sample. We tested one flow at a time, so we could not measure fairness or BBR's effect on competing flows. The bottleneck location and buffer size are unknown, and our VM on Wi-Fi can add jitter. True capacity BDP is unknown. TCP_INFO sampling every 0.2 s averages short bursts. These measurements use BBRv1; its loss behavior and results must not be generalized to other BBR versions. The original archive contains per-run summaries and plotted traces, but not the original interval CSVs; its pooled quantiles cannot be independently reconstructed from summaries.

Separate Docker rerun: four destinations, two repetitions

On October 21, the full pipeline was rerun with:

```
./scripts/docker.sh run -n 4 -d 20 -i 0.1 -r 2 --algorithm cubic --seed 42
```

The manifests record Linux `6.18.50`; the representative metadata records x86_64 and little-endian TCP_INFO decoding. Part 1 completed 4/4 destinations in five attempts after `iperf.online.net:5209` refused negotiation. Part 3 reused those four host/port pairs, used 20 s transfers and 0.1 s sampling for every algorithm, and completed 24/24 tests, with no skipped algorithms or failed Part 3 attempts. Its two orders were CUBIC–Reno–BBR and Reno–BBR–CUBIC. Full raw traces, server results, inputs, metadata, and PDFs are preserved in `rerun-results/`.

Table 3: Separate rerun: minimum, median, and p95 pool interval rates across the two runs; mean and sample SD summarize per-run means. RTT averages the per-run mean smoothed RTTs. All rates are Mbit/s.

Destination	Algo	Runs	Min	Median	Mean	SD	P95	RTT (ms)
LeaseWeb US	CUBIC	2	0.00	198.84	189.39	38.06	299.83	61.8
LeaseWeb US	Reno	2	0.00	217.52	208.16	64.68	324.25	73.7
LeaseWeb US	BBR	2	0.00	199.74	189.20	27.77	298.35	55.8
Multinet24 PL	CUBIC	2	0.00	116.30	98.12	4.56	185.75	141.0
Multinet24 PL	Reno	2	0.00	162.87	151.76	6.72	192.56	136.4
Multinet24 PL	BBR	2	0.00	157.44	146.96	1.63	180.87	138.6
Scaleway NL	CUBIC	2	0.00	117.71	107.33	46.51	204.24	120.5
Scaleway NL	Reno	2	0.00	202.74	190.67	14.60	225.33	110.3
Scaleway NL	BBR	2	0.00	179.46	159.61	5.20	208.99	120.5
Mauritius Tel.	CUBIC	2	0.00	9.81	24.78	0.85	91.11	305.4
Mauritius Tel.	Reno	2	0.00	62.92	62.38	2.20	157.79	303.4
Mauritius Tel.	BBR	2	0.00	60.60	62.35	1.18	178.03	294.5

Reno had the largest mean of per-run goodputs on all four destinations, although its Mauritius result was effectively tied with BBR (62.38 vs. 62.35 Mbit/s). BBR had lower grouped mean RTT than both other algorithms on LeaseWeb and Mauritius; on Multinet24, Reno had the lowest mean RTT. On the representative Multinet24 path, all six Part 3 tests recorded zero retransmitted segments. CUBIC averaged about 1,424 segments, Reno 4,019, and BBR 5,611, with mean RTTs of 141.0, 136.4, and 138.6 ms. This illustrates why a larger cwnd alone need not imply higher goodput: pacing, actual flight size, startup, and other limits also matter.

Most rerun tests had no retransmissions, so a universal loss-limited interpretation would be unsupported. Within-destination goodput coefficients of variation ranged from 1.1% to 43.3% across the two repetitions. Two short tests are insufficient to establish steady-state behavior or a reliable universal ranking. Host/kernel, duration, sampling, destinations, and measurement time differ from the earlier batch; we cannot attribute the ranking change to any one factor. Both batches support the need to discuss variability and qualify operating-regime claims.

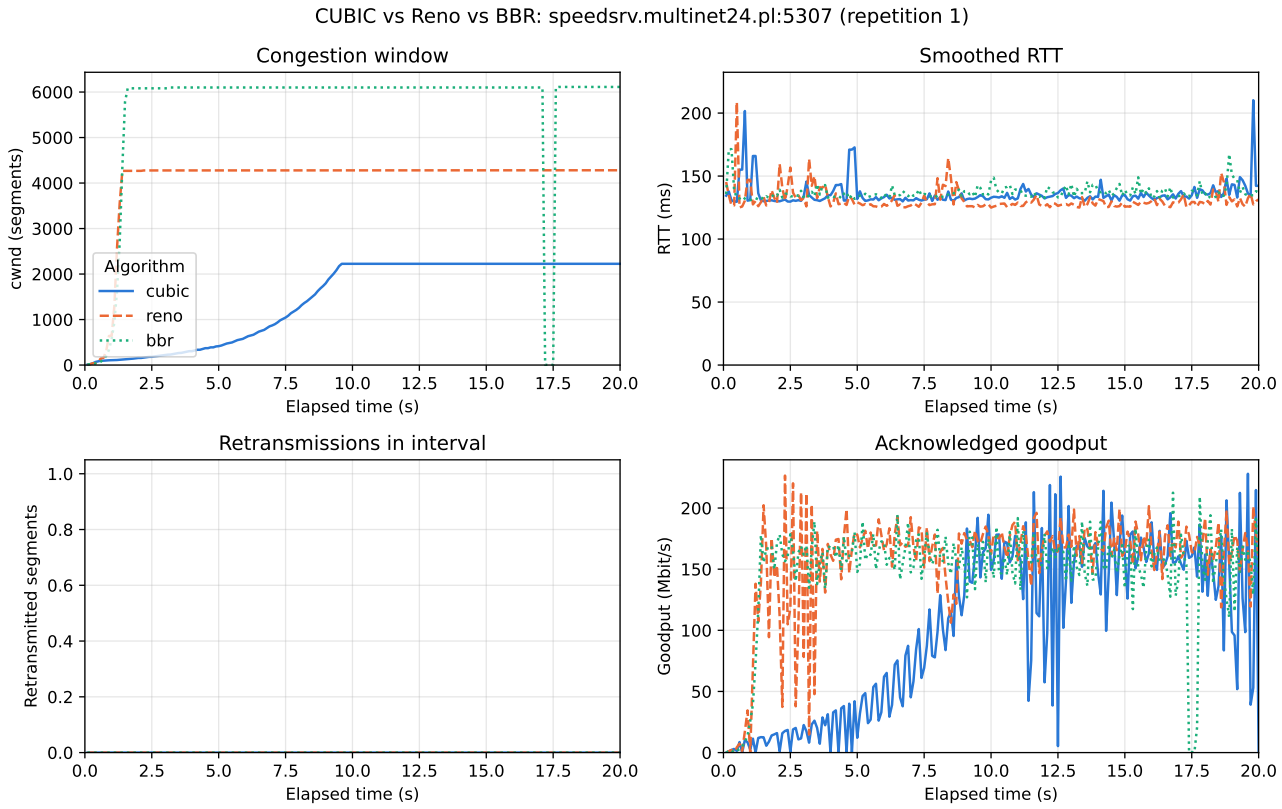


Figure 4: Separate rerun: shared-scale time series at `speedsrv.multinet24.pl:5307`, repetition 1.

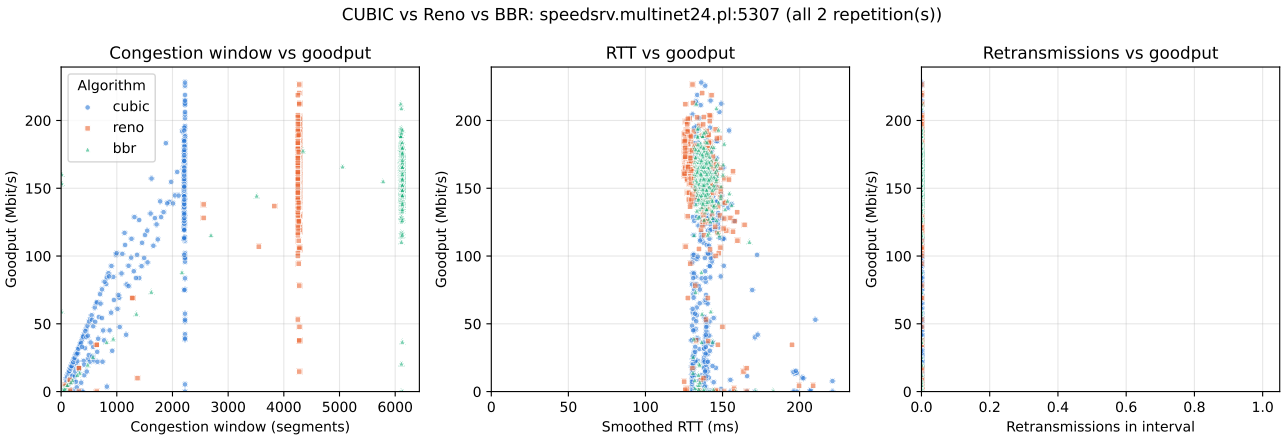


Figure 5: Separate rerun: cwnd, RTT, and interval retransmissions against goodput, both repetitions at the same representative destination.

Reproduction and code

From `Assignment2/` on Linux with `reno cubic bbr` available:

```
python3 main.py -n 5 -d 60 -i 0.2 -r 3 --seed 42
```

Raw traces go to `output/part3/`. Earlier-batch summaries, logs, and plots are in `Assignment2/part3a-results/`; the separate rerun's complete measurements are in `Assignment2/rerun-results/`. The implementation cited below is byte-for-byte unchanged in the inspected rebased checkout at `e89808a`.

- Experiment loop (Part 1 destinations, rotating order, retries): `run_part3`; per-run metrics: `path_metrics`; CLI options: `main.py` L45–96.
- Algorithm selection and check: `run_test.py` L151–171; goodput and loss proxy: `compute_sample`.
- Plots: `plot_comparison` (per-algorithm pages are also in `part3a-results/comparison.pdf`).

Use of AI tools

We used Claude Code (Anthropic) to help set up the Linux VM, implement the Part 3 changes, and draft this writeup from our measurement files. We reviewed the code, tested it with short runs before the full experiment, and checked the numbers against `part3a-results/`. An OpenAI coding assistant also helped recheck references and calculations, qualify the interpretations, and prepare the separate rerun comparison.